

Chapter 1

Assignment 2: Semantic Extraction

Building on the syntactic pipeline of Assignment 1, this stage extracts *semantic* information from each clause: named entities that identify the parties, values, and legal references involved; semantic roles that describe who does what to whom; and an intent label that captures the functional type of the clause.

1.1 Custom Named Entity Recognition (NER)

1.1.1 Entity Schema

Six domain-specific entity types are defined for legal contract NER:

- **PARTY** — contracting parties and organisations (e.g. *Party A*, *ABC International Co.*, *Ltd.*).
- **MONEY** — monetary amounts with or without currency symbols (e.g. *USD 10,000*, *\$5,000*).
- **DATE** — absolute and relative date expressions (e.g. *05 May 2024*, *within 30 days*).
- **RATE** — percentage or per-unit rates (e.g. *1% per month*, *2% per annum*).
- **PENALTY** — penalty clauses and liquidated-damages expressions.
- **LAW** — statutory references, ordinances, and legal codes (e.g. *Civil Code 2015*).

The full BIO label set used during model training is:

O, B-PARTY, I-PARTY, B-MONEY, I-MONEY, B-DATE, I-DATE, B-RATE, I-RATE,
B-PENALTY, I-PENALTY, B-LAW, I-LAW

1.1.2 Model Architecture

Primary model — Legal-BERT fine-tuned NER. The primary model fine-tunes `nlpaueb/legal-bert-base-uncased` [1] for token classification using the HuggingFace Trainer API. Key training hyperparameters are summarised in Table 1.1.

Table 1.1: Legal-BERT NER training configuration.

Hyperparameter	Value
Base model	nlpaueb/legal-bert-base-uncased
Epochs	3
Learning rate	2×10^{-5}
Batch size	8
Train/eval split	90 / 10
Evaluation metric	sequeval (entity-level F1)

Fallback — Rule-based NER. When Legal-BERT confidence falls below a configurable threshold, a `RuleBasedNER` module is invoked. It applies hand-crafted regular expressions tailored to each entity type — for example, monetary patterns of the form `(USD|VND|\$)\s*[\d,]+` for MONEY, and ISO-format or natural-language date patterns for DATE.

1.1.3 Output Format

Results are written to `output/ner_results.json` as a list of objects:

```

1  [
2    {
3      "clause_id": 0,
4      "text": "Party A shall pay Party B USD 10,000 by 05 May
5        2024...",
6      "entities": [
7        {"text": "Party A",      "label": "PARTY",  "start": 0, "
8          end": 7},
9        {"text": "Party B",      "label": "PARTY",  "start": 16, "
10         end": 23},
11        {"text": "USD 10,000",   "label": "MONEY",   "start": 24, "
12         end": 34},
13        {"text": "05 May 2024", "label": "DATE",    "start": 38, "
14         end": 49},
15        {"text": "1% per month", "label": "RATE",    "start": 53, "
16         end": 65}
17      ]
18    }
19  ]

```

Listing 1.1: NER output format (abbreviated).

1.1.4 NER Example

Table 1.2 shows the entities extracted from a representative legal clause.

Table 1.2: Entities extracted from “*Party A shall pay Party B USD 10,000 by 05 May 2024 at 1% per month penalty rate.*”

Entity Text	Label	Start	End
Party A	PARTY	0	7
Party B	PARTY	16	23
USD 10,000	MONEY	24	34
05 May 2024	DATE	38	49
1% per month	RATE	53	65

1.2 Semantic Role Labelling (SRL)

1.2.1 Approach

Rather than a dedicated BERT-based SRL model, the system uses a *dependency-tree-based* approach built on top of spaCy’s `en_core_web_sm` parser. For each clause, the ROOT token (typically the main verb) is identified as the predicate, and semantic roles are assigned to its dependents by mapping Universal Dependencies labels to PropBank-style argument roles, as shown in Table 1.3.

Table 1.3: Mapping from Universal Dependencies labels to semantic roles.

UD Dependency Label	Semantic Role	Notes
nsubj	Agent	Active-voice subject
nsubjpass	Theme	Passive-voice subject
dobj / obj	Theme	Direct object
iobj	Recipient	Indirect object
prep (to-)	Recipient	Prepositional recipient
Temporal prepositions	Time	prep with time head
advcl + cond. mark	Condition	Subordinate conditional clause

1.2.2 Output Format

Results are written to `output/srl_results.json` as a list of objects with the following structure:

```

1  [
2    {
3      "clause_id": 0,
4      "text": "The Employer shall pay the Employee the monthly
           salary ...",
5      "predicate": "pay",
6      "roles": {
7        "Agent": "The Employer",
8        "Theme": "the monthly salary",
9        "Recipient": "the Employee",
10       "Time": "within 30 days"
11     }

```

```

12   }
13 ]

```

Listing 1.2: SRL output format.

1.2.3 SRL Example

Input:

“The Employer shall pay the Employee the monthly salary within 30 days.”

Output:

Predicate: *pay*
 Agent: *The Employer*
 Theme: *the monthly salary*
 Recipient: *the Employee*
 Time: *within 30 days*

The ROOT token *pay* is the predicate. *The Employer* is the active nominal subject (**nsubj**) mapped to Agent; *the monthly salary* is the direct object (**dobj**) mapped to Theme; *the Employee* is reached via the indirect object (**iobj**) relation mapped to Recipient; and *within 30 days* is a temporal prepositional phrase mapped to Time.

1.3 Intent Classification

1.3.1 Task Definition and Label Set

Intent classification assigns each clause one of four functional intent labels:

- **Obligation** — a party is required to perform an action (keywords: *shall, must, is obligated to*).
- **Prohibition** — an action is forbidden (keywords: *shall not, must not, is prohibited*).
- **Right** — a party is entitled to perform an action (keywords: *may, has the right to, is entitled to*).
- **Termination Condition** — the contract terminates upon an event (keywords: *terminates, expires, is terminated upon/if*).

A priority order — Prohibition > Termination Condition > Right > Obligation — is used to resolve clauses matching multiple keyword patterns during weak-label generation.

1.3.2 Model 1 — TF-IDF + Logistic Regression

Training labels are generated by a keyword-regex weak labeller. A TF-IDF vectoriser converts each clause to a feature vector, and a Logistic Regression classifier is trained on these vectors. Table 1.4 summarises the configuration.

Table 1.4: TF-IDF + Logistic Regression configuration.

Component	Setting
<i>TfidfVectorizer</i>	
n-gram range	(1, 3)
max features	5 000
sublinear TF	True
<i>LogisticRegression</i>	
Regularisation C	1.0
Solver	lbfgs
Multi-class	multinomial
Max iterations	1 000

1.3.3 Model 2 — Fine-tuned BERT

The second classifier fine-tunes `bert-base-uncased` as a sequence classifier using `AutoModelForSequenceClassification`. Table 1.5 summarises training settings.

Table 1.5: BERT intent classification training configuration.

Hyperparameter	Value
Base model	<code>bert-base-uncased</code>
Epochs	3
Learning rate	2×10^{-5}
Max token length	256

1.3.4 Output Format

Results are written to `intent_classification.txt` as a TSV file with columns `clause_text`, `tfidf_label`, `bert_label`, and `confidence`.

1.3.5 Comparative Example

Table 1.6 compares the predictions of both models on representative clauses.

Table 1.6: Intent classification results for sample clauses.

Clause Excerpt	TF-IDF	BERT	Conf.
<i>“... shall pay...”</i>	Obligation	Obligation	0.96
<i>“... shall not disclose...”</i>	Prohibition	Prohibition	0.98
<i>“... may terminate...”</i>	Right	Termination condition	0.72
<i>“... terminates upon breach...”</i>	Termination condition	Termination condition	0.95

The “*may terminate*” clause illustrates a common ambiguity: TF-IDF’s n-gram features weight *may* heavily toward the Right class, while BERT’s contextual represen-

tation captures the termination semantics of the full phrase and predicts Termination Condition. The lower confidence score (0.72) correctly signals the model’s uncertainty on this boundary case.

Bibliography

- [1] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, *LEGAL-BERT: The Muppets straight out of Law School*, in *Findings of EMNLP 2020*, pp. 2898–2904, 2020.